

# Scraper de la Consulta Pública del PJe TRF5

Cómo funciona el código, qué hubo que descubrir del portal y cómo defender cada decisión en la entrevista final.

**106.763**

PROCESOS EN EL CORPUS

**3.432**

LÍNEAS DE TYPESCRIPT

**56**

TESTS, 8 SUITES, OFFLINE

**0**

NAVEGADORES USADOS

El proyecto recorre **todos** los procesos de `pjett.trf5.jus.br`, extrae los metadatos completos de cada uno y descarga los PDF de sus documentos. Solo `axios` + `cheerio`: sin Puppeteer, sin Playwright, sin Selenium. El portal es JSF 1.2 + RichFaces sobre JBoss Seam, no navega por URL y no tiene paginación, así que casi todo el diseño sale de entender su protocolo antes de escribir una línea de scraping.

## CÓMO USAR ESTA GUÍA

Las secciones 1–8 son el recorrido del código: léelas con el repositorio abierto al lado. La sección 9 reúne las capturas de terminal de la corrida real. La sección 10 es el banco de preguntas: **si solo tienes una hora, estudia la 10 y vuelve a las demás para sustentar cada respuesta**. La sección 11 es la chuleta de cifras y comandos para repasar cinco minutos antes.

## ESTADO EN EL MOMENTO DE ESCRIBIR LA GUÍA · 2026-08-26 06:13 UTC

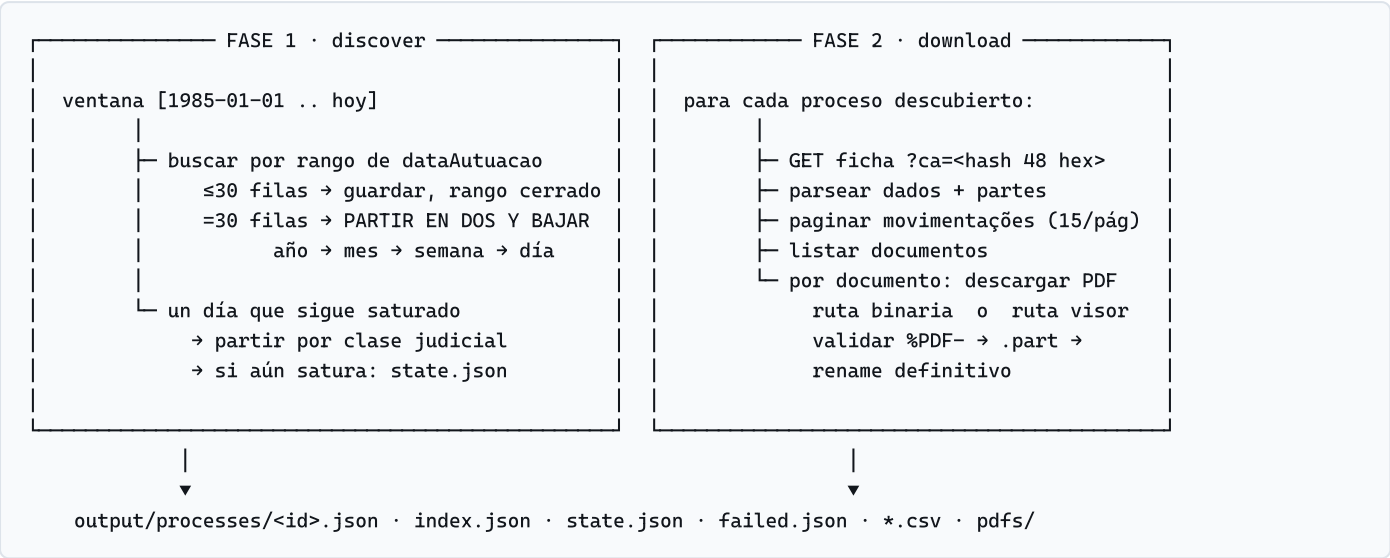
Corrida completa en curso: **10.062 procesos** descubiertos de 106.763, **123 fichas** enriquecidas, **1.101 documentos** catalogados, rango `1985-01-01 .. 2020-06-11` cerrado, 1 día saturado, 5 fallos anotados para reintento. Todas las capturas de esta guía son salida verbatim de esa corrida o de comandos ejecutados sobre ella.

- 1 El mapa de 30 segundos
- 2 Estructura de carpetas: qué hace cada fichero
- 3 El portal por dentro: JSF, RichFaces y el botón señuelo
- 4 Fase 1 · descubrimiento: particionar en vez de paginar
- 5 Fase 2 · enriquecimiento: ficha, partes y movimentações
- 6 Descarga y validación de los PDF
- 7 Errores, reintentos y backoff
- 8 Persistencia y reanudación
- 9 Capturas de terminal de la corrida real
- 10 Banco de preguntas de entrevista con respuestas



# 1 · El mapa de 30 segundos

Si te piden «explícame tu scraper» y tienes un minuto, esto es lo que dices.



## Las tres frases que lo resumen todo

1. **El botón de buscar es un señuelo.** Su `onclick` es `return ejecutarReCaptcha();;A4J.AJAX.Submit(...)`: el `return` corta antes del submit visible. El submit real es un `a4j:jsFunction` oculto, `ejecutarPesquisa`, cuyo id cambia entre despliegues y se lee de la página en cada sesión.
2. **No hay paginación.** Toda consulta devuelve como máximo 30 filas y el hueco «Paginação» se renderiza vacío. «Navegar todas las páginas» se convierte en **particionar el espacio de búsqueda** por fecha de autuação hasta que cada tramo cabe bajo el tope.
3. **Los enlaces de descarga caducan con la sesión.** La ficha se abre con un hash `ca` estable, pero los enlaces de sus documentos llevan un hash ligado a la sesión que abrió la ficha. Por eso cada PDF se descarga partiendo de una ficha recién abierta, nunca de URLs guardadas.

## Las capas y por qué existen

| CAPA                  | RESPONSABILIDAD                                         | POR QUÉ ESTÁ SEPARADA                                                                                        |
|-----------------------|---------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| <code>http/</code>    | Transporte: cookies, charset, delays, tipado de errores | Todo lo que sabe de la red vive aquí. El resto del código nunca ve un status HTTP crudo, ve un error tipado. |
| <code>pje/</code>     | Protocolo del portal: A4J, sesión, parsers, descarga    | Es la parte que se rompe si el tribunal cambia el HTML. Aislada y cubierta con fixtures reales.              |
| <code>crawl/</code>   | Estrategia: qué pedir, en qué orden, cuándo parar       | No sabe de HTML ni de HTTP. Solo decide la partición y el orden de enriquecimiento.                          |
| <code>storage/</code> | Persistencia atómica, índice, estado, fallos            | Único punto que toca el disco. Hace posible cortar con Ctrl+C y reanudar sin duplicados.                     |
| <code>util/</code>    | Reintentos, fechas, texto, ids, logger, rename          | Funciones puras: es lo que la suite de tests puede fijar sin red.                                            |

#### LA REGLA DE DISEÑO QUE ATRAVIESA TODO EL PROYECTO

**Nunca completar algo en silencio.** Si una búsqueda devuelve un mensaje del portal en vez de una tabla, el rango *no* se marca como completado. Si las movimentações no se pudieron paginar hasta el final, el proceso queda con `detailFetched: false`. Si un día sigue saturado tras partir por clase, se anota en `state.json.saturatedDays`. Un hueco explícito se reintenta; un hueco silencioso se convierte en datos incorrectos que nadie detecta.

## 2 · Estructura de carpetas: qué hace cada fichero

```
PowerShell - scraper-challenge

PS C:\Users\js\Desktop\scraper-challenge> tree /f src

src/
├─ index.ts           115 líneas  CLI: discover / download / all / retry-failed / status
├─ config.ts          107 líneas  toda la configuración por variable de entorno
├─ types.ts           151 líneas  contrato de datos: ProcessRecord, DocumentRecord...
├─ http/
│   └─ client.ts       214 líneas  axios + cookies + charset + delays + tipado de errores
├─ pje/
│   ├── a4j.ts          157 líneas  protocolo Ajax4jsf: serializar, construir, parchear
│   ├── session.ts      203 líneas  sesión JSF: abrir, buscar, ficha, páginas de movimentações
│   ├── listParser.ts   155 líneas  tabla de resultados: 30 filas, tope, segredo de justiça
│   ├── detailParser.ts 263 líneas  ficha: dados, partes, movimentações, documentos
│   └─ documents.ts     152 líneas  descarga de PDF por dos rutas + validación
├─ crawl/
│   ├── discover.ts     233 líneas  FASE 1: partición por fechas + medición del total
│   └─ enrich.ts        265 líneas  FASE 2: fichas completas + descargas
├─ storage/
│   └─ store.ts         460 líneas  shards por proceso, índice, estado, fallos, CSV
├─ util/
│   ├── retry.ts        147 líneas  taxonomía de errores + backoff exponencial
│   ├── dates.ts        92 líneas  ISO → dd/MM/yyyy, splitRange, aritmética en UTC
│   ├── text.ts         60 líneas  ids estables, CNJ, nombres de fichero seguros
│   ├── fs.ts           30 líneas  rename con reintento (bloqueo transitorio de Windows)
│   └─ logger.ts        55 líneas  líneas con marca de tiempo a stdout y a fichero
└─ __tests__/
    ├── fixtures/
    │   ├── landing.html 46 KB · detail-process.html 106 KB
    │   ├── document-viewer.html 52 KB · search-results.xml 35 KB
    │   └─ *.test.ts
    │       a4j 8 · dates 6 · detailParser 7 · fs 3 · listParser 8
    │       retry 15 · store 4 · text 5
```

**Captura 1.** Árbol real del proyecto con el recuento de líneas de cada módulo (3.432 en total).

### Recorrido en el orden en que se ejecutan

| FICHERO                          | QUÉ HACE Y QUÉ TIENES QUE SABER DECIR DE ÉL                                                                                                                                                                                                                                                                                       |
|----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>index.ts</code>            | Punto de entrada. Elige comando, monta el <code>Store</code> , registra <code>SIGINT</code> / <code>SIGTERM</code> para guardar antes de morir y despacha a <code>Discoverer</code> o <code>Enricher</code> . <code>status</code> no toca la red: imprime el progreso leyendo los ficheros de salida.                             |
| <code>config.ts</code>           | Un solo objeto <code>CONFIG</code> congelado ( <code>as const</code> ), poblado desde variables de entorno con validación ( <code>envInt</code> revienta si le pasas algo que no es un número $\geq 0$ ). Ninguna otra parte del código lee <code>process.env</code> .                                                            |
| <code>http/client.ts</code>      | Lo que axios no trae: tarro de cookies manual, decodificación por charset, pausa mínima con jitter, y sobre todo <code>classify()</code> , que convierte statuses y páginas de error disfrazadas de 200 en errores tipados <b>antes</b> de que nadie parsee nada. <code>maxRedirects: 0</code> porque aquí un 302 significa algo. |
| <code>pje/a4j.ts</code>          | El protocolo RichFaces en tres funciones: <code>serializeForm</code> (el formulario entero, en orden de documento, como haría el navegador), <code>buildA4jBody</code> (los marcadores que añade A4J) y <code>applyA4jResponse</code> (parchear el documento vivo con los fragmentos que devuelve el servidor).                   |
| <code>pje/session.ts</code>      | La conversación con el portal: <code>open()</code> lee cookies, ViewState y el control de búsqueda oculto; <code>search()</code> hace el POST A4J; <code>getDetail(ca)</code> abre la ficha; <code>getMovementsPage()</code> mueve el slider de movimentações.                                                                    |
| <code>pje/listParser.ts</code>   | Convierte la tabla de resultados en filas. Devuelve además <code>isCapped</code> y <code>overflowBanner</code> : las dos señales que gobiernan la partición.                                                                                                                                                                      |
| <code>pje/detailParser.ts</code> | Ficha completa. Todo se localiza por <b> sufijo </b> de id ( <code>[id\$=":processoEvento"]</code> ) porque los prefijos <code>j_idNNN</code> cambian entre despliegues.                                                                                                                                                          |
| <code>pje/documents.ts</code>    | Las dos rutas de descarga y <code>validatePdf()</code> . Escribe <code>.part</code> y renombra solo tras validar.                                                                                                                                                                                                                 |
| <code>crawl/discover.ts</code>   | La estrategia de cobertura: pila de rangos, partición binaria, partición secundaria por clase, medición del total y marcado de días saturados.                                                                                                                                                                                    |
| <code>crawl/enrich.ts</code>     | Fase 2. Aísla el fallo por elemento: un documento que agota reintentos no tumba el proceso, y un proceso que falla no tumba la corrida.                                                                                                                                                                                           |
| <code>storage/store.ts</code>    | Un fichero JSON por proceso ( <i>shard</i> ) escrito atómicamente, más un índice ligero. Los shards son la fuente de verdad; el índice se reconstruye desde ellos.                                                                                                                                                                |
| <code>util/retry.ts</code>       | La taxonomía de errores y <code>withRetry</code> . Es el fichero que más te van a preguntar.                                                                                                                                                                                                                                      |

#### DETALLE QUE SUENA BIEN EN UNA ENTREVISTA

`util/fs.ts` existe por un fallo real: en Windows, renombrar sobre un fichero que el indexador o el antivirus tiene abierto unos milisegundos falla con `EPERM` / `EBUSY`. Toda escritura atómica del scraper termina en un rename, y una fase de descubrimiento de 19 minutos murió exactamente ahí. El fix es un rename con reintento breve (25 ms, 50 ms, 100 ms...) solo para esos tres códigos de error; cualquier otro sigue propagándose. Es un ejemplo de «el bug me enseñó dónde estaba el supuesto equivocado», no de defensa preventiva.

## 3 · El portal por dentro: JSF, RichFaces y el botón señuelo

Esta sección responde la pregunta «¿cómo enfrentaste JSF?». Es el corazón técnico del proyecto.

### 3.1 Qué es exactamente el portal

| PIEZA                                               | CONSECUENCIA PRÁCTICA PARA EL SCRAPER                                                                                                           |
|-----------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| JBoss Seam 2 + JSF 1.2                              | El estado de la vista vive en el servidor. Cada interacción exige devolver <code>javax.faces.ViewState</code> y el formulario <b>completo</b> . |
| RichFaces / Ajax4jsf 3.3                            | No hay navegación por URL: todo es un POST por XHR y la respuesta es un XML que dice qué fragmentos del DOM reemplazar.                         |
| Ids <code>j_id244</code> , <code>j_id561</code> ... | Los genera el framework y cambian entre despliegues. <b>Nunca se codifican a mano</b> : se leen de la página o se buscan por sufijo.            |
| HTML en ISO-8859-1, XML en UTF-8                    | Decodificar con el charset que declara cada respuesta o los acentos de «Movimentaçoões» se corrompen.                                           |
| WAF F5 delante                                      | Sirve una página de bloqueo con <b>HTTP 200</b> . Hay que detectarla por contenido, no por status.                                              |
| reCAPTCHA cargado pero <b>desactivado</b>           | Compila a <code>if (false) { grecaptcha.execute() }</code> . No hay captcha que resolver ni que evadir.                                         |

### 3.2 El botón señuelo

Este es el hallazgo que desbloqueó todo. El `onClick` del botón visible «Pesquisar» es:

```
onClick="return ejecutarReCaptcha();A4J.AJAX.Submit('fPP',...,{ 'parameters':{'fPP:searchProcessos':...}});return false;

    ▲ devuelve undefined ... y el `return` sale de la función AQUÍ.
    El A4J.AJAX.Submit que ves nunca llega a ejecutarse.

// El submit real es un a4j:jsFunction declarado en otra parte de la página:
ejecutarPesquisa=function(){A4J.AJAX.Submit('fPP',null,{...,'parameters':{'fPP:j_id244':'fPP:j_id244'}})};
```

Enviar `fPP:searchProcessos` devuelve un 200 perfectamente válido... que solo re-renderiza el panel de mensajes. Enviar el control del `jsFunction` renderiza la tabla de resultados. Ese id cambia entre despliegues, así que `findSearchControl()` lo extrae de la página en cada apertura de sesión y cae al botón visible solo si no lo encuentra:

```
// src/pje/session.ts - findSearchControl()
const at = html.indexOf('ejecutarPesquisa=function');
if (at >= 0) {
    const pairs = parseA4jParameters(html.slice(at, at + 1200));
    const own = pairs.find(([k, v]) => k === v); // el control se nombra a sí mismo
    if (own) return own;
}
```

### 3.3 El ciclo de una petición A4J

- |                                          |                                                                                                                                                                                                |
|------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. <code>serializeForm(\$, 'fPP')</code> | → todos los input/select/textarea del formulario, en orden de documento                                                                                                                        |
| 2. <code>overrides</code>                | → se sustituyen los campos de criterio POR SUFIJO (:dataAutuacaoInicioInputDate...)                                                                                                            |
| 3. <code>buildA4jBody</code>             | → <code>AJAXREQUEST=_viewRoot + campos + &lt;control&gt;=&lt;control&gt; + AJAX:EVENTS_COUNT=1</code>                                                                                          |
| 4. <code>POST listView.seam</code>       | → <code>Content-Type: application/x-www-form-urlencoded; charset=UTF-8</code>                                                                                                                  |
| 5. <code>respuesta XML</code>            | → <code>&lt;meta name="Ajax-Update-Ids" content="fPP:processosGridPanel"/&gt;</code><br>→ <code>&lt;div id="ajax-view-state"&gt;&lt;input name="javax.faces.ViewState" value="..."/&gt;</code> |

```
6. applyA4jResponse($, xml)    → reemplaza esos ids en el documento VIVO y refresca el ViewState
7. parseListPage($)           → se lee la tabla del documento ya parcheado
```

El punto 6 es la idea clave: el scraper mantiene un documento cheerio vivo y lo parchea **igual que haría el navegador**. Sin eso, la segunda búsqueda enviaría un ViewState caducado.

#### LA TRAMPA QUE EVITA DATOS INCORRECTOS

Una respuesta A4J que solo actualiza el panel de mensajes es un 200 válido y deja la tabla **anterior** en el documento. Si la parseas, completas un rango con filas de otra consulta. Por eso `search()` exige que los ids actualizados incluyan `processosGridPanel` o `processosTable`; si no, lanza `UnexpectedStructureError` con el mensaje del portal.

### 3.4 Sesión, cookies y expiración

- Node no tiene tarro de cookies: se implementa a mano. Cookies observadas: `JSESSIONID`, `ROUTER_ID` y dos del WAF (`trf50...`).
- La cabecera `Cookie:` solo se envía si hay algo que enviar: una cabecera `Cookie:` vacía es uno de los disparadores del bloqueo del WAF.
- La sesión se recicla cada `SESSION_MAX_REQUESTS` (400 peticiones) antes de que el portal la tire él.
- `ViewExpiredException` en el XML → `SessionExpiredError` → se reabre sesión y se reintenta.
- `open()` pone `this.$ = undefined` **antes** de resetear las cookies: si la reapertura falla, la sesión queda cerrada. Dejar el documento viejo haría que el siguiente intento enviara un ViewState antiguo sin cookies.



## 4 · Fase 1 · descubrimiento: particionar en vez de paginar

### 4.1 El problema: el portal miente sobre su tamaño

- Toda consulta filtrada devuelve **como máximo 30 filas**.
- El pie dice `min(total, 30) resultados encontrados`, así que el número que ves nunca pasa de 30.
- El hueco del paginador (`<div title="Paginação">`) se renderiza **vacío siempre**. No hay «página 2» que pedir.

Conclusión: «navegar por todas las páginas del sitio» no se puede hacer paginando. Se hace **particionando el espacio de búsqueda** hasta que cada partición cabe bajo el tope.

### 4.2 La medición del volumen — el test en frío, sin ingesta

#### CÓMO SE MIDió EL CORPUS

El formulario rechaza una búsqueda sin criterios («Pelo menos um dos critérios de pesquisa deve ser informado»). Pero una fecha **sintácticamente válida e inexistente**, `31/02/2000`, pasa la validación y el conversor de JSF la descarta: la consulta corre **sin filtro** y el pie anuncia el total real del corpus. Es una sola petición, se lanza **antes de guardar nada** y sus filas **no se ingieren**: solo se lee el número. Medición en frío, coste 1 petición: **106.763 procesos**.

```
// src/crawl/discover.ts - measureTotal(): se ejecuta al principio de run(), antes del primer rango
const page = await this.session.search({ fromDate: '31/02/2000', dateTo: '31/02/2000' });
if (page.announcedTotal > CONFIG.resultCap) {
  this.store.setMeasuredTotal(page.announcedTotal); // solo el número. Ninguna fila se almacena.
  log.info(`portal total: ${page.announcedTotal} processes (unfiltered count)`);
}
```

El total se cachea en `state.json` con su marca de tiempo y se vuelve a medir si tiene más de 24 horas. `npm run status` lo contrasta contra lo descubierto: por eso el progreso se lee como `10062 of 106763 announced by the portal` y no como un porcentaje inventado.

```
output/scrapper.log - arranque de la corrida

[2026-08-26T05:19:08.746Z + 3.9s] INFO command=all base=https://pjett.trf5.jus.br output=...\output
[2026-08-26T05:19:08.917Z + 4.1s] INFO discovery window 1990-01-01..1994-12-31 (1826 days), result cap 30
[2026-08-26T05:19:12.213Z + 7.4s] INFO session opened: cookies=[JSESSIONID, ROUTER_ID, trf501ad1ee3, trf501f66e06] viewState=j_id1 searchControl=fPP:j_id244
[2026-08-26T05:19:13.644Z + 8.8s] INFO portal total: 106763 processes (unfiltered count)
[2026-08-26T05:19:14.827Z + 10.0s] INFO search 1990-01-01..1994-12-31: 7 rows (7 new) | total stored 7
```

**Captura 2.** La medición en frío ocurre en el segundo 8,8 de la corrida, con la sesión recién abierta y cero procesos almacenados. Fíjate en `searchControl=fPP:j_id244`: el id oculto leído de la página, no codificado a mano.

### 4.3 El algoritmo de partición

```
pila = [ {1985-01-01 .. hoy} ]
mientras la pila no esté vacía:
  rango = pila.pop()
  si el rango ya está en state.json.completedRanges → saltar ← esto es lo que hace la fase reanudable
  resultado = buscar(rango)
    ≤ 29 filas → guardar filas, markRangeCompleted(rango)
    = 30 filas (CAPPED) → si el rango dura > 1 día: [a, b] = splitRange(rango); pila.push(b, a)
                        si el rango es 1 solo día: splitDayByClass(día)
```

- **Partición binaria, no calendario.** `splitRange` parte el rango por la mitad en días. Se adapta sola a la densidad: 1986–2004 caben por año (147 procesos en total), 2005–2026 bajan hasta mes/semana/día.

- **Orden cronológico.** Se hace `push(b, a)` para que `pop()` saque primero la mitad temprana; el log se lee en orden y es reanudable de forma intuitiva.
- **Partición secundaria por clase judicial.** Si un solo día sigue devolviendo 30, se relanza la consulta filtrando por cada clase que aparece **en la propia respuesta saturada** (siempre hay material, porque cada fila imprime su clase).
- **Días saturados.** Si aun así queda residuo **y** el portal mostró el banner de desborde, el día se anota en `state.json.saturatedDays` con una nota. Es cobertura parcial declarada.

```

output/scraper.log - partición binaria en acción

+ 2.9s INFO search 1985-01-01..2026-08-26: 30 rows (25 new) CAPPED -> split | total stored 32
+ 4.4s INFO search 1985-01-01..2005-10-28: 30 rows (19 new) CAPPED -> split | total stored 51
+ 6.5s INFO search 1995-06-01..2005-10-28: 30 rows (11 new) CAPPED -> split | total stored 62
+ 7.6s INFO search 1995-06-01..2000-08-13: 30 rows ( 0 new) CAPPED -> split | total stored 62
+ 9.9s INFO search 1998-01-06..2000-08-13: 30 rows (24 new) CAPPED -> split | total stored 86
+ 13.2s INFO search 2000-08-14..2005-10-28: 30 rows (30 new) CAPPED -> split | total stored 118

```

**Captura 3.** 41 años → 20 años → 10 → 5 → 2. El `(0 new)` de la cuarta línea es normal: los rangos se solapan durante el descenso y el índice deduplica por id.

```

output/scraper.log - un día saturado y su partición por clase

+1086.4s INFO search 2019-12-20..2019-12-20: 30 rows (8 new) CAPPED -> split | total stored 8764
+1086.4s WARN day 2019-12-20 is capped at 30: splitting by 5 class name(s) seen in the capped response
+1086.9s INFO search 2019-12-20..2019-12-20 class="APELAÇÃO / REMESSA NECESSÁRIA": 10 rows (0 new)
+1087.8s INFO search 2019-12-20..2019-12-20 class="APELAÇÃO CRIMINAL": 2 rows (0 new)
+1088.8s INFO search 2019-12-20..2019-12-20 class="APELAÇÃO CÍVEL": 19 rows (3 new)
+1089.5s INFO search 2019-12-20..2019-12-20 class="AGRAVO REGIMENTAL CÍVEL": 1 rows (0 new)
+1090.3s INFO search 2019-12-20..2019-12-20 class="CUMPRIMENTO DE SENTENÇA": 1 rows (0 new)
+1091.4s INFO search 2019-12-21..2019-12-21: 11 rows (3 new) | total stored 8770

```

**Captura 4.** El día 20/12/2019 devolvía 30 filas; las 5 clases visibles suman 33 filas y ninguna satura, así que el día queda cubierto. Esta es la única situación en toda la corrida que exigió partición secundaria.

#### MATIZ QUE DEMUESTRA QUE ENTENDISTE EL PROBLEMA

30 filas **no** significan necesariamente desbordamiento: un día con exactamente 30 procesos está completo. Por eso el parser distingue `isCapped` (llegó al tope, hay que investigar) de `overflowBanner` (el portal dijo textualmente «somente os 30 primeiros serão exibidos»). Solo el banner es prueba de que se ocultaron filas, y solo el banner marca el día como saturado.

## 5 · Fase 2 · enriquecimiento: ficha, partes y movimentações

### 5.1 Qué se extrae de cada proceso

| SECCIÓN DE LA FICHA           | CÓMO SE LOCALIZA                                                              | NOTAS                                                                                                                  |
|-------------------------------|-------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| Dados do Processo             | <code>.propertyView</code> → pares <code>.name</code> / <code>.value</code>   | Se guarda tal cual, con las etiquetas del portal como claves.                                                          |
| Polo ativo / passivo / outros | <code>table[id\$=":processoPartesPoloAtivoResumidoList"]</code><br>y hermanas | Los abogados van indentados: se detectan por ausencia de <code>span.text-bold</code> → <code>isRepresentative</code> . |
| Movimentações                 | <code>table[id\$=":processoEvento"]</code>                                    | <b>15 por página.</b> Se paginan todas con el slider (ver 5.2).                                                        |
| Documentos                    | <code>table[id\$=":processoDocumentoGridTab"]</code>                          | Cada fila publica su ruta de descarga (binaria o visor).                                                               |

### 5.2 La paginación de movimentações: un slider, no un paginador

Las movimentações se paginan con un `rich:inputNumberSlider` que vive en su **propio formulario** y en su **propia región Ajax**. El scraper lee del JavaScript inline el número de páginas (`'maxValue'`), el control que dispara el `onchange` y el `containerId`:

```
// detailParser.ts - findMovementsPager() lee esto del HTML:
new Richfaces.Slider("X:j_id561:j_id562",{ 'minValue':'1', 'maxValue':'7', ...
  'onchange':'A4J.AJAX.Submit(\'X:j_id561\',event,{...\''containerId\':'\'X:j_id474\','
    '\parameters\':{\''X:j_id561:j_id563\':'\'X:j_id561:j_id563\'}}}')

// session.ts - getMovementsPage(): el cuerpo del POST para la página N
const body = buildA4jBody($detail, { formId: pager.formId, control: pager.control },
  [[pager.pageField, String(page)]]);
if (pager.containerId) body[0] = ['AJAXREQUEST', pager.containerId];
```

Verificado en la corrida real con historiales de 90, 282, 343 y 597 movimentações, todos completos.

#### REGLA: UN HISTORIAL TRUNCADO NO ES UNA FICHA COMPLETA

Si alguna página del slider falla, el proceso se guarda con lo recogido pero con `detailFetched: false` y una entrada en `failed.json` que dice `movements truncated: 120 of 282 collected`. La siguiente corrida lo rehace. Marcarlo como completo sería registrar un historial parcial como si fuera el historial entero: el peor error posible en datos judiciales.

### 5.3 Dos optimizaciones que conviene saber explicar

- **El HTML de la ficha se vuelve a pedir en cada pasada** (los enlaces de documento están ligados a la sesión), **pero el historial ya completo no se vuelve a paginar**. En un proceso de 597 movimentações eso ahorra 39 POST A4J en la pasada de PDFs.
- **Un proceso cuya ficha ya agotó sus intentos se deja para `retry-failed`** en vez de reintentarse en cada reanudación: si no, cada arranque paga el calendario completo de backoff por un proceso que probablemente sigue roto.

```
output/scrapper.log - cola de la corrida en curso

+ 2.1s INFO detail BR-TRF5-0000121-52.2012.4.05.8303: 2 parties, 158/158 movements, 8 documents
+ 2.9s INFO detail BR-TRF5-0000122-42.2009.4.05.8400: 4 parties, 78/78 movements, 2 documents
+ 4.1s INFO detail BR-TRF5-0000122-75.2014.4.05.8106: 9 parties, 343/343 movements, 10 documents
+ 9.9s INFO detail BR-TRF5-0000122-88.1990.8.15.0351: 4 parties, 156/156 movements, 13 documents
+ 30.6s INFO detail BR-TRF5-0000123-60.2009.4.05.8001: 12 parties, 597/597 movements, 15 documents
+ 36.8s INFO detail BR-TRF5-0000125-91.2009.4.05.8401: 12 parties, 175/175 movements, 10 documents
+ 52.6s INFO detail BR-TRF5-0000130-84.2016.4.05.8202: 18 parties, 307/307 movements, 15 documents
+ 78.1s INFO detail BR-TRF5-0000134-78.2017.4.05.8108: 12 parties, 410/410 movements, 15 documents
```

**Captura 5.** 597/597 significa recogidas / anunciadas por el portal. El proceso de 597 movimentações tardó ~20 s: son 39 POST del slider a ~0,7 s cada uno más la ficha. Ritmo observado: ~6,7 s por ficha completa.

5.4 Estructura de un registro guardado

```
{
  "id": "BR-TRF5-0006273-97.1992.4.05.0000",    // PAIS-FUENTE-clave única de la fuente
  "source": "BR-TRF5",
  "number": "0006273-97.1992.4.05.0000",      // número CNJ
  "ca": "8a8adae773b623ea...",               // hash de acceso a la ficha (48 hex, estable)
  "className": "CUMPRIMENTO DE SENTENÇA CONTRA A FAZENDA PÚBLICA",
  "distributionDate": "1992-05-26",
  "details": { "Número Processo": "...", "Jurisdição": "TRF5", ... },
  "parties": [ { "pole": "ATIVO", "name": "CLEONICE MARINHO DE ANDRADE (ESPÓLIO)",
                 "role": "REQUERENTE", "isRepresentative": false },
               { ..., "role": "ADVOGADO", "document": "OAB MS4120-B - CPF: 048.769.891-68",
                 "isRepresentative": true } ],    // 18 partes en total
  "movementsTotal": 282, "movements": [ ... ],  // 282/282 recogidas
  "documents": [ { "id": "...-DOC-6052946", "idBin": "5989146", "title": "Despacho",
                  "download": "binary", "status": "downloaded",
                  "file": "pdfs\\..._6052946_2025-08-15_Despacho.pdf", "bytes": 20051 } ],
  "detailFetched": true,
  "firstSeenAt": "2026-08-26T05:19:14.636Z", "updatedAt": "2026-08-26T05:42:38.702Z"
}
```

## 6 · Descarga y validación de los PDF

### 6.1 Dos rutas, ambas implementadas

```
RUTA A · binaria (documentos guardados como binario: Decisão, Inteiro Teor, Acórdão...)
GET listView.seam?idBin=N&...&actionMethod=...setDownloadInstance(row)
→ 302 Location: /pjeconsulta/download.seam?cid=N
GET /pjeconsulta/download.seam?cid=N
→ 200 application/pdf Content-Disposition: filename="..." bytes %PDF-1.4

RUTA B · visor (documentos nacidos como HTML: Despacho, Certidão...)
GET documentoSemLoginHTML.seam?ca=<hash 96 hex>&idProcessoDoc=N → visor HTML
POST documentoSemLoginHTML.seam
campos del formulario del visor + <form>:downloadPDF + ca + idProcDocBin
→ 200 application/pdf attachment; filename="<numeroCNPJ>_<idDoc>.pdf"

RUTA C · ninguna la fila no publica control de descarga → status: "unavailable"
```

Qué ruta usa cada documento lo dice la propia fila: si trae `href` con `setDownloadInstance` es binaria; si trae `documentoSemLoginHTML.seam` es visor; si no trae ninguna, se registra como no disponible y **no** como fallo.

#### EL DETALLE QUE OBLIGA A REDISEÑAR LA FASE 2

El hash `ca` de la **ficha** (48 hex) es **estable entre sesiones** — verificado abriéndolo en una sesión virgen. El hash `ca` de un **documento** (96 hex) está **ligado a la sesión** que abrió la ficha: reutilizarlo en otra sesión da 302. Por eso el descargador nunca usa URLs guardadas: cada PDF parte de una ficha recién abierta en la misma sesión. Y si un intento falla por sesión, el reintento vuelve a leer los enlaces de una ficha nueva antes de repetir.

### 6.2 La validación: qué significa «es un PDF»

```
// src/pje/documents.ts
export function validatePdf(res: HttpResponse): void {
  if (res.status !== 200) throw new InvalidDownloadError(`HTTP ${res.status} instead of a PDF`);
  const magic = res.body.subarray(0, 5).toString('latin1'); // ← bytes mágicos
  if (magic !== '%PDF-') {
    const kind = /text\/html/i.test(res.contentType) ? 'an HTML page'
      : `content-type ${res.contentType}`;
    throw new InvalidDownloadError(`server returned ${kind} instead of a PDF (${res.body.length} B)`);
  }
  if (res.body.length < MIN_PDF_BYTES) throw new InvalidDownloadError(`PDF too small`); // 200 B
}

// y la escritura, solo DESPUÉS de validar:
fs.writeFileSync(`${target}.part`, res.body);
renameSyncWithRetry(`${target}.part`, target); // rename atómico
```

Tres comprobaciones independientes: **status**, **firma** `%PDF-` **en los primeros 5 bytes** y **tamaño mínimo**. El content-type se usa para redactar el error, no para decidir: un servidor puede mentir en la cabecera, los bytes no. Y el fichero se escribe como `.part` y se renombra solo al final, así que una caída nunca deja medio PDF con aspecto de PDF completo.

```
PowerShell - verificación de los PDF en disco

PS> find output/pdfs -name "*.pdf" | wc -l
11

PS> for f in $(find output/pdfs -name "*.pdf"); do printf "%-6s %8s B  %s\n" "$(head -c 5 $f)" "$(stat -c%s $f)"
"$${basename $f}"; done
%PDF-      3487 B  BR-TRF5-0000005-48.2009.4.05.8401_11788020_2026-07-31_Despacho.pdf
%PDF-     61197 B  BR-TRF5-0000007-07.1985.8.20.0124_3469065_2025-10-27_Acordao.pdf
%PDF-     36044 B  BR-TRF5-0000007-07.1985.8.20.0124_7222997_2026-05-11_Acordao.pdf
%PDF-     20032 B  BR-TRF5-0003216-56.1994.4.05.8001_7907537_2020-10-18_Despacho.pdf
%PDF-     20152 B  BR-TRF5-0003216-56.1994.4.05.8001_7907550_2021-01-20_Decisao.pdf
%PDF-    245706 B  BR-TRF5-0006273-97.1992.4.05.0000_6052692_2023-06-30_CUMSENA8-RN_027_AR...pdf
%PDF-    123563 B  BR-TRF5-0006273-97.1992.4.05.0000_6052700_2023-06-30_CUMSENA8-RN_055_Des...pdf
%PDF-     21562 B  BR-TRF5-0006273-97.1992.4.05.0000_6052785_2023-09-15_Decisao.pdf
```

**Captura 6.** Los 11 PDF descargados empiezan por `%PDF-`. La comprobación de la terminal es exactamente la que hace `validatePdf()` en tiempo de ejecución.

## 6.3 El nombre del fichero

```
output/pdfs/<idProceso>/<idProceso>_<idDocumento>_<fecha>_<título>.pdf
```

```
BR-TRF5-0006273-97.1992.4.05.0000_6052785_2023-09-15_Decisao.pdf
└── proceso ───────────┬── doc ┬── fecha ┬── título ┘
```

`safeFileName()` quita acentos (NFD + eliminación de diacríticos), deja solo `[A-Za-z0-9._-]` y acota la longitud, así que el mismo nombre funciona en Windows, macOS y Linux. Es **descriptivo** (se sabe qué es sin abrirlo) y **estable** (el mismo documento produce siempre el mismo nombre, lo que permite saltarse la descarga si ya existe con tamaño válido).

## 7 · Errores, reintentos y backoff

El fichero más preguntado del proyecto: [src/util/retry.ts](#), 147 líneas, 15 tests.

### 7.1 La regla: clasificar primero, reaccionar después

| SEÑAL                                                                 | TIPO                     | REACCIÓN                                                                                         |
|-----------------------------------------------------------------------|--------------------------|--------------------------------------------------------------------------------------------------|
| 429                                                                   | HttpRetryableError       | Backoff exponencial con jitter; si viene <code>Retry-After</code> , manda el servidor.           |
| 500 502 503 504 408                                                   | HttpRetryableError       | Igual que el 429.                                                                                |
| Cortes de red ( <code>ECONNRESET</code> , <code>ETIMEDOUT</code> ...) | código de error de Node  | Reintento con backoff.                                                                           |
| 200 + «Requisição - Rejeitada»                                        | WafBlockedError          | Pausa de 90 s y <b>sesión nueva</b> . Reintentar rápido convierte un bloqueo blando en uno duro. |
| 302 → <code>errorUnexpected.seam</code>                               | HttpRetryableError(503)  | Pausa de 5 s y reintento <b>sin</b> reabrir sesión (la sesión está bien, el servidor no).        |
| <code>ViewExpiredException</code> / 302 a la portada                  | SessionExpiredError      | Reabrir sesión y repetir.                                                                        |
| HTML donde iba un PDF                                                 | InvalidDownloadError     | <b>No</b> se guarda basura con extensión <code>.pdf</code> ; se anota y se sigue.                |
| Página con estructura desconocida                                     | UnexpectedStructureError | Nunca se reintentan: el HTML no va a cambiar por insistir.                                       |
| 403 / 404                                                             | HttpFatalError           | Fatal. El servidor ya dijo que no; repetir solo añade carga.                                     |

### 7.2 El backoff

```
// src/util/retry.ts
export function backoffMs(attempt: number, err?: unknown): number {
  // 1. Una instrucción explícita del servidor manda por encima del calendario propio.
  if (err instanceof HttpRetryableError && err.retryAfterMs !== undefined)
    return Math.min(err.retryAfterMs, RETRY_AFTER_HARD_CAP_MS); // tope de cordura: 15 min

  // 2. El WAF tiene su propia pausa, mucho más larga.
  if (err instanceof WafBlockedError) return CONFIG.retry.wafCooldownMs; // 90 s

  // 3. Exponencial con FULL JITTER: base·2^(n-1), con techo, y luego la mitad + azar.
  const exp = CONFIG.retry.baseDelayMs * 2 ** Math.max(0, attempt - 1); // 2s, 4s, 8s...
  const capped = Math.min(exp, CONFIG.retry.maxDelayMs); // techo 120 s
  return Math.round(capped / 2 + Math.random() * (capped / 2)); // jitter
}
```

El jitter no es decorativo: sin él, varios elementos que fallan a la vez reintentan a la vez y golpean el servidor en el mismo instante. Con jitter completo cada espera cae en `[capped/2, capped]`, así que los reintentos se dispersan.

### 7.3 El bucle

```

for (let attempt = 1; attempt <= max; attempt++) {
  try { return await fn(attempt); }
  catch (err) {
    lastErr = err;
    // se sale si el error no es reintentable, si el llamante lo veta, o si era el último intento
    if (!isRetryable(err) || (opts.retryIf && !opts.retryIf(err)) || attempt === max) break;
    await sleep(backoffMs(attempt, err));
    if (opts.onRetry) {
      try { await opts.onRetry(err, attempt + 1); }
      catch (hookErr) {
        // El hook de recuperación hace red (reabrir sesión) y puede caer él mismo en el
        // throttle. Eso debe consumir el presupuesto de ESTE reintento, no abortar el bucle
        // ni atribuir mal el fallo. Su propio Retry-After / cooldown se respeta antes de seguir.
        await sleep(backoffMs(attempt, hookErr));
      }
    }
  }
}
throw lastErr; // el llamante lo anota en failed.json y continúa con el siguiente

```

#### TRES DECISIONES DEL BUCLE QUE VALEN UNA PREGUNTA CADA UNA

1. `onRetry` reabre la sesión **solo** si el error es de sesión o del WAF: un 429 corriente no debe costar una petición extra a la portada de un servidor que ya está saturado.
2. El presupuesto de intentos es **por elemento**, no global: cada búsqueda, cada ficha y cada documento tiene su propio `withRetry`. Un proceso roto no consume los intentos de los demás.
3. `retryIf` permite vetos locales. En la paginación de movimentações se veta el reintento ante `SessionExpiredError`: las páginas siguientes necesitan justamente el ViewState que acaba de morir, así que reintentar sobre el mismo documento es tirar peticiones.

## 7.4 Cuando se agotan los intentos

1. El elemento se anota en `output/failed.json` con clave, fase, motivo, número de intentos acumulado **entre corridas**, marca de tiempo y status HTTP.
2. **La corrida continúa con el siguiente elemento.** El bucle de fase 2 tiene `try/catch` por ficha y por documento.
3. Un éxito posterior limpia la entrada (`clearFailure`), así que `failed.json` refleja lo que sigue roto, no el historial.
4. `npm run retry-failed` reprocesa solo esas claves.



## 8 · Persistencia y reanudación

### 8.1 Por qué un JSON monolítico no servía

#### EL CÁLCULO QUE FUERZA EL DISEÑO

106.763 procesos × decenas de KB por registro enriquecido. Reescribir un único `processes.json` tras cada proceso es  $O(n^2)$  de E/S, y su forma serializada supera el tamaño máximo de string de Node mucho antes de terminar el corpus. No es una preferencia de estilo: el monolito **no puede funcionar** a esta escala.

| FICHERO                                                 | ¿RECONSTRUIBLE?                 | PAPEL                                                                                                                 |
|---------------------------------------------------------|---------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <code>processes/&lt;id&gt;.json</code>                  | — <b>es la fuente de verdad</b> | Un fichero por proceso, escrito atómicamente en el momento en que el registro cambia. $O(1)$ por actualización.       |
| <code>index.json</code>                                 | Sí, desde los shards            | Índice ligero para dedupe, iteración y progreso. Se vacía a disco cada 60 s / 500 actualizaciones, no en cada cambio. |
| <code>state.json</code>                                 | <b>No</b>                       | Rangos completados, días saturados, total medido. Es el punto de reanudación.                                         |
| <code>failed.json</code>                                | <b>No</b>                       | Qué falló, por qué y cuántas veces.                                                                                   |
| <code>processes.csv</code> / <code>documents.csv</code> | Sí                              | Exportación tabular UTF-8 con BOM (Excel conserva los acentos).                                                       |

Lo no reconstruible se trata distinto: si `state.json` existe pero no se puede leer, se aparta con marca de tiempo y **la corrida aborta**. Empezar de cero en silencio perdería primero el punto de reanudación y después sobrescribiría la evidencia.

### 8.2 Reanudación tras un corte

```
Ctrl+C      → SIGINT: store.save(true) + exportCsv() + exit 130      (guardado limpio)
kill / apagón → los shards ya están en disco; index.json puede ir atrasado
arranque     → loadIndex() lee index.json ... y luego reconcileShards():
               todo shard que el índice no conoce se añade al índice
               «index.json was behind the shard files: N process(es) recovered»
fase 1       → los rangos de state.json.completedRanges se saltan
fase 2       → solo se procesan procesos sin detailFetched o con documentos pendientes
```

Ese `reconcileShards()` es la red de seguridad para el caso feo: matar el proceso sin SIGINT. Sin él, los procesos escritos en los últimos 60 segundos serían invisibles para la fase 2, para el CSV y para `status` — aunque su fichero estuviera perfectamente en disco.

### 8.3 Deduplicación

Los rangos se solapan durante el descenso binario, así que el mismo proceso se lista muchas veces. El `upsert` por id resuelve todos los casos:

```
const existing = this.get(record.id);
merged = existing
  ? { ...existing, ...stripUndefined(record), // lo nuevo definido gana...
      documents: mergeDocuments(existing.documents, record.documents),
      firstSeenAt: existing.firstSeenAt, // ...pero la 1ª vez no se pierde
      updatedAt: now }
  : { ...record, firstSeenAt: record.firstSeenAt || now, updatedAt: now };
return !existing; // true solo si era nuevo → "(N new)" del log
```

- **Re-listar no deshace el enriquecimiento:** `storeRow()` conserva `detailFetched` y `foundInRange` del registro existente. Si no, cada solape forzaría volver a bajar la ficha entera.
- **Una descarga completada nunca se degrada:** `mergeDocuments` mantiene `downloaded` + `file` + `bytes` aunque el nuevo registro venga como `pending`, y un éxito borra el `error` de un fallo anterior.
- **El shard manda sobre el índice:** `get()` lee del disco aunque el índice en memoria no conozca el id, y de paso cura el índice. Así un índice desactualizado nunca provoca que se sobrescriba un shard en lugar de fusionarlo.

#### LIMITACIÓN HONESTA QUE CONVIENE RECONOCER TÚ MISMO

`npm run status` lee `index.json`, que se vacía a disco cada 60 s / 500 actualizaciones. Ejecutado **desde otro proceso** mientras la corrida está viva, sus contadores por proceso pueden ir por detrás de los shards: en la captura de esta guía informa de 8 PDF descargados cuando en disco ya hay 11. Los datos están completos; el resumen es el que va atrasado. `reconcileShards()` hoy solo añade procesos que faltan, no refresca los contadores de los que ya conoce.

## 9 · Capturas de terminal de la corrida real

Todo lo de esta sección es salida verbatim, capturada el 2026-08-26 sobre la corrida en curso. Son la evidencia que respalda cada respuesta de la sección 10.

```
PowerShell - npm run status

PS C:\Users\js\Desktop\scraper-challenge> npm run status

> scraper-challenge@1.0.0 status
> ts-node src/index.ts status

[2026-08-26T06:13:38.105Z + 0.4s] INFO command=status base=https://pjett.trf5.jus.br
output=C:\Users\js\Desktop\scraper-challenge\output attempts=3
processes discovered : 10062 of 106763 announced by the portal
details fetched      : 123
documents            : 1101 (downloaded 8, pending 1093, failed 0, unavailable 0)
completed ranges     : 1 (1985-01-01 .. 2020-06-11)
saturated days       : 1
failures to retry    : 5
```

**Captura 7.** El comando de progreso. No toca la red: lo deduce de los ficheros de salida. Contrasta lo descubierto contra el total medido, así que el avance es verificable y no una estimación.

```
PowerShell - npm test

PS C:\Users\js\Desktop\scraper-challenge> npm test

> scraper-challenge@1.0.0 test
> jest

[... WARN test: attempt 1/3 failed (HTTP 429): HTTP 429 -> waiting 1s
[... WARN test: attempt 2/3 failed (HTTP 429): HTTP 429 -> waiting 3s
[... WARN test: attempt 1/3 failed (HTTP 429): HTTP 429 -> waiting 30s ← Retry-After manda
[... WARN test: attempt 1/5 failed (HTTP 503): HTTP 503 -> waiting 1s
[... WARN test: attempt 2/5 failed (HTTP 503): HTTP 503 -> waiting 2s
[... WARN test: attempt 3/5 failed (HTTP 503): HTTP 503 -> waiting 4s ← exponencial con jitter
[... WARN test: attempt 1/3 failed: The JSF session or view state expired -> waiting 2s
[... WARN test: recovery before attempt 2 failed too: HTTP 429 -> waiting 1s ← el hook también falla

Test Suites: 8 passed, 8 total
Tests:       56 passed, 56 total
Snapshots:   0 total
Time:        9.297 s
Ran all test suites.
```

**Captura 8.** La suite corre **sin red**, con timers falsos (las esperas de segundos se completan al instante) y con fixtures reales del portal. Los WARN son las políticas de reintento ejercitándose, no fallos.

```
PowerShell - npm run typecheck

PS C:\Users\js\Desktop\scraper-challenge> npm run typecheck

> scraper-challenge@1.0.0 typecheck
> tsc --noEmit && tsc --noEmit -p tsconfig.test.json

(sin salida = sin errores)
```

**Captura 9.** TypeScript en modo `strict`, con `noUncheckedIndexedAccess` y `noImplicitOverride`. Se comprueban también los tests, que usan un tsconfig propio.

```
output/scraper.log - reintentos y agotamiento sobre una ficha rota
```

```
[05:19:21.641Z + 16.8s] WARN detail BR-TRF5-0006051-07.1991.4.05.8200: attempt 1/5 failed (HTTP 503): portal error
page (errorUnexpected) for detail ?ca=77b2370dc7ad... -> waiting 20s
[05:19:42.242Z + 37.4s] WARN detail BR-TRF5-0006051-07.1991.4.05.8200: attempt 2/5 failed (HTTP 503): portal error
page (errorUnexpected) ... -> waiting 20s
[05:20:02.768Z + 57.9s] WARN detail BR-TRF5-0006051-07.1991.4.05.8200: attempt 3/5 failed (HTTP 503): portal error
page (errorUnexpected) ... -> waiting 20s
[05:20:23.334Z + 78.5s] WARN detail BR-TRF5-0006051-07.1991.4.05.8200: attempt 4/5 failed (HTTP 503): portal error
page (errorUnexpected) ... -> waiting 20s
[05:20:43.858Z + 99.0s] ERROR detail BR-TRF5-0006051-07.1991.4.05.8200 failed permanently: portal error page
(errorUnexpected) for detail ?ca=77b2370dc7ad...
                                     ↑ 5 intentos, 99 s de reloj. La corrida NO se detuvo: siguió con el proceso
siguiente.

- tras bajar MAX_ATTEMPTS de 5 a 3 (la pausa sigue en 20 s: el proceso vivo arrancó antes de ese cambio) -

[05:56:00.639Z + 1.6s] WARN detail BR-TRF5-0000017-65.2018.4.05.8201: attempt 1/3 failed (HTTP 503): portal error
page (errorUnexpected) ... -> waiting 20s
[05:56:21.341Z + 22.3s] WARN detail BR-TRF5-0000017-65.2018.4.05.8201: attempt 2/3 failed (HTTP 503): portal error
page (errorUnexpected) ... -> waiting 20s
[05:56:42.115Z + 43.0s] ERROR detail BR-TRF5-0000017-65.2018.4.05.8201 failed permanently: portal error page
(errorUnexpected) ...
```

**Captura 10.** El mismo error, antes y después de bajar los intentos de 5 a 3 con la evidencia recogida: 99 s de bloqueo por proceso roto pasan a 43 s. La pausa de `errorUnexpected` también se bajó de 20 s a 5 s, pero eso solo se ve en las líneas de un proceso arrancado después del cambio: el de la captura sigue con 20 s. Ver pregunta 1 de la sección 10.

```
output/failed.json

{
  "BR-TRF5-0000017-65.2018.4.05.8201": {
    "key": "BR-TRF5-0000017-65.2018.4.05.8201",
    "stage": "detail",
    "reason": "portal error page (errorUnexpected) for detail ?ca=ae6734c4ec53...",
    "attempts": 2,
    "lastAttemptAt": "2026-08-26T05:56:42.115Z",
    "httpStatus": 503
  },
  ... 4 entradas más, todas errorUnexpected en la misma fase ...
}
```

**Captura 11.** 5 fallos sobre 123 fichas (4 %), todos del mismo tipo y todos reintentables con `npm run retry-failed`. Un éxito posterior borra la entrada.

```
output/scrapper.log - descargas de PDF

+107.3s INFO PDF pdfs\BR-TRF5-0006273-97.1992.4.05.0000\..._6052785_2023-09-15_Decisao.pdf (21562 B)
+108.9s INFO PDF pdfs\BR-TRF5-0006273-97.1992.4.05.0000\..._6052808_2023-06-30_CUMSENF8-
RN_077_Inteiro_Teor_do_Acordao...pdf (232371 B)
+110.0s INFO PDF pdfs\BR-TRF5-0006273-97.1992.4.05.0000\..._6052692_2023-06-30_CUMSENF8-RN_027_AR_ref._of._2465...pdf
(245706 B)
+111.3s INFO PDF pdfs\BR-TRF5-0006273-97.1992.4.05.0000\..._6052700_2023-06-30_CUMSENF8-RN_055_Despacho_-_Oficio...pdf
(123563 B)
```

**Captura 12.** Una descarga cada ~1,3 s: es la pausa de cortesía (700 ms + hasta 300 ms de jitter) más el tiempo del servidor. Nombres descriptivos, carpeta por proceso.

output/state.json - el punto de reanudación

```
{
  "completedRanges": [ { "from": "1985-01-01", "to": "2020-06-11" } ],
  "saturatedDays": [ {
    "day": "2019-12-20", "classSplitDone": true,
    "note": "all classes seen in the response are covered; classes hidden past the cap cannot be queried"
  } ],
  "updatedAt": "2026-08-26T05:44:15.034Z",
  "measuredTotal": 106763,
  "measuredAt": "2026-08-26T05:19:13.644Z"
}
```

**Captura 13.** 35 años de rangos fusionados en un solo intervalo cerrado: `mergeRanges()` coalesce los adyacentes, así que el estado no crece sin control. El día saturado queda documentado con su motivo.

## 10 · Banco de preguntas de entrevista

Cada pregunta trae **respuesta corta** (lo que dices en 20 segundos) y **sustento** (lo que sacas si te piden detalle). Las preguntas 1 a 7 son las que ya sabes que van a caer.

### 1 ¿Cómo escalaste los intentos?

**Respuesta corta:** exponencial con jitter completo —  $2s \cdot 2^n$ , techo de 120 s, 3 intentos por elemento — pero con tres excepciones que **no** siguen esa curva: `Retry-After` del servidor manda siempre, el bloqueo del WAF tiene una pausa fija de 90 s y la página de error del portal tiene 5 s. Y los números no son por defecto: los bajé con la evidencia de la corrida.

#### Sustento

- **Presupuesto por elemento, no global.** Cada búsqueda, cada ficha y cada documento tiene su propio `withRetry`. Un proceso roto no consume los intentos de los demás.
- **La curva:**  $base \cdot 2^{(n-1)}$  con techo, y luego jitter completo ( $capped/2 + azar \cdot capped/2$ ). Sin jitter, todo lo que falla junto reintentará junto.
- **Jerarquía de prioridades:** instrucción explícita del servidor (`Retry-After`, con tope de cordura de 15 min) > pausa específica del tipo de bloqueo > curva exponencial.
- **Cómo llegué a 3 intentos.** Empecé en 5 con 20 s de pausa para `errorUnexpected.seam`. Al medirlo vi que ese error es **determinista por proceso**: el mismo 302 en sesión nueva 15 minutos después, con los procesos vecinos funcionando. No es rate limiting, es un proceso que ese endpoint no sabe servir. Con 5 intentos  $\times$  20 s, cada proceso roto bloqueaba 99 s de reloj para nada. Lo bajé a **3 intentos** (efecto ya medido: 43 s en vez de 99 s) y la pausa de 20 s a **5 s**, que sigue cubriendo un agotamiento de pool genuinamente transitorio. **El 429 conserva su curva completa**, que es donde la espera larga sí sirve.
- **Cuando se agotan:** se anota en `failed.json` con el contador acumulado **entre corridas** y la corrida sigue. `npm run retry-failed` los reprocesa después.

**EVIDENCIA** Captura 10: la misma ficha, 5 intentos / 99 s antes, 3 intentos / 43 s después. **TESTS**

`retry.test.ts` fija el calendario, el `Retry-After`, el techo y el agotamiento.

## 2 ¿Qué aplica cuando la fuente está rota?

**Respuesta corta:** depende de **qué** se rompió. Clasifico el fallo antes de reaccionar y hay cuatro reacciones distintas: esperar, reabrir sesión, marcar fatal, o marcar **hueco explícito**. Lo único que nunca hago es dar por bueno un resultado incompleto.

### Sustento — el árbol de decisión completo

| CÓMO ESTÁ ROTA                                                                         | QUÉ APLICO                                                                                                                                                             |
|----------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Sobrecargada (429, 5xx, corte de red)                                                  | Esperar. Backoff exponencial, <code>Retry-After</code> si lo manda.                                                                                                    |
| Me bloqueó el WAF (200 disfrazado)                                                     | Pausa larga de 90 s + <b>sesión nueva</b> . Insistir rápido convierte el bloqueo blando en duro.                                                                       |
| Sesión/vista caducada                                                                  | Reabrir sesión y repetir. Es recuperable y barato.                                                                                                                     |
| Un proceso concreto que el portal no sabe servir ( <code>errorUnexpected.seam</code> ) | 3 intentos cortos, luego <code>failed.json</code> y seguir. Insistir no lo arregla.                                                                                    |
| Me devuelve HTML donde iba un PDF                                                      | No guardar nada. <code>InvalidDownloadError</code> → el documento queda <code>failed</code> , no queda un <code>.pdf</code> corrupto en disco.                         |
| 404 / 403                                                                              | Fatal, sin reintento. El servidor ya respondió.                                                                                                                        |
| <b>Cambió el HTML</b> y el parser no reconoce la página                                | <code>UnexpectedStructureError</code> , <b>nunca reintentado</b> , con el detalle en el mensaje. Es un fallo de código, no de red: reintentar solo oculta el problema. |
| Respondió, pero con un mensaje en vez de resultados                                    | El rango <b>no se marca completado</b> y se anota. Es el caso más peligroso: un 200 perfectamente válido que dejaría un hueco permanente.                              |
| Un día tiene más procesos de los que la interfaz deja ver                              | <code>state.json.saturatedDays</code> con la nota del motivo: cobertura parcial <b>declarada</b> .                                                                     |
| Mi propio estado está corrupto ( <code>state.json</code> ilegible)                     | Apartarlo con marca de tiempo y <b>abortar</b> . Empezar de cero en silencio perdería el punto de reanudación y luego sobrescribiría la evidencia.                     |

**El principio:** un hueco explícito se puede reintentar; un hueco silencioso se convierte en datos incorrectos que nadie detecta. Todo el diseño de errores sale de ahí.

### 3 ¿Cómo definiste el volumen de la fuente?

**Respuesta corta:** lo **medí**, no lo estimé. Con un **test en seco y en frío, sin ingesta**: una sola petición al arrancar, antes de guardar un solo registro, que fuerza al portal a declarar el total. Resultado: **106.763 procesos**. Todo el diseño de la fase 1 sale de ese número.

#### Sustento — por qué hizo falta un truco

1. El portal **no publica** su total: toda consulta filtrada devuelve como máximo 30 filas y el pie anuncia `min(total, 30)`. El paginador se renderiza vacío. Preguntando de forma normal nunca ves más de «30 resultados encontrados».
2. Una búsqueda sin criterios la rechaza la validación: «Pelo menos um dos critérios de pesquisa deve ser informado».
3. La grieta: una fecha **sintácticamente válida pero inexistente**, `31/02/2000`, pasa la validación de «al menos un criterio» y el conversor de JSF la descarta. La consulta corre **sin filtro** y el pie anuncia el total real del corpus.

#### Por qué es «en frío» y «sin ingesta»

- **En frío:** es la primera búsqueda de la corrida, con la sesión recién abierta y con cero procesos almacenados. En el log ocurre en el segundo 8,8 (Captura 2).
- **Sin ingesta:** de esa respuesta **solo se lee el número del pie**. Ninguna de sus 30 filas se guarda. El código lo dice literalmente: «*Purely informational*». No contamina el corpus con filas de una consulta que no corresponde a ningún rango.
- **Coste: 1 petición.** Se cachea en `state.json` con su marca de tiempo y solo se vuelve a medir si tiene más de 24 horas.

#### Para qué sirve el número, más allá de la cifra

- Es el **denominador del progreso**: `npm run status` dice `10062 of 106763 announced by the portal`. Sin él, «llevo 10.000» no significa nada.
- Dimensiona la arquitectura: 106.763 registros de decenas de KB es lo que **descarta** el JSON monolítico y obliga a un fichero por proceso (sección 8.1).
- Permite estimar el coste de la corrida completa antes de lanzarla (pregunta 11).

#### SI TE PREGUNTAN «¿Y SI ESE NÚMERO MIENTE?»

Es un límite superior anunciado por el propio portal, no una verdad auditada. Por eso la cobertura **no** se demuestra con él: se demuestra con `state.json.completedRanges`, que es el registro de qué intervalos de fecha se cerraron por debajo del tope, más `saturatedDays`, que es la lista explícita de dónde la interfaz pública no deja llegar. El total medido es el contraste, no la prueba.



## 4 ¿Validaste que es un PDF? ¿Abriste la fuente a mano?

**Respuesta corta:** sí a las dos. Valido con **tres comprobaciones** (status, firma `%PDF-` en los primeros bytes, tamaño mínimo) y escribo a `.part` antes de renombrar. Y sí: recorrí el portal a mano petición por petición antes de escribir el scraper — el protocolo está documentado en `docs/protocolo.md` y las respuestas reales están versionadas como fixtures de los tests.

### La validación

- **No me fío del content-type.** Un servidor puede mandar `application/pdf` y devolver una página de error. Los bytes no mienten: compruebo `%PDF-` en los primeros 5. El content-type solo lo uso para redactar un error legible.
- **Tamaño mínimo** de 200 B: descarta respuestas vacías o truncadas.
- **Escritura `.part` + rename.** Una caída a mitad de escritura nunca deja un fichero con extensión `.pdf` y contenido incompleto. El rename es atómico.
- Si algo falla, el documento queda `status: "failed"` con el motivo y se anota en `failed.json`. **Nunca se guarda basura con extensión de PDF.**

**EVIDENCIA** Captura 6: los 11 PDF en disco empiezan por `%PDF-`, entre 3 KB y 245 KB.

### Abrir la fuente a mano

- Todo el protocolo se capturó en vivo el 2026-08-25/26 y está escrito petición a petición en `docs/protocolo.md`: cuerpos de POST, cabeceras, redirecciones y respuestas. Lo no verificado está marcado como tal — por ejemplo `reportPDF.seam` («Imprimir», expediente completo), que devolvió 302 en las dos rutas probadas y por eso no se usa.
- Hay experimentos deliberados, no solo observación: probé el hash `ca` de una ficha en una **sesión virgen** para comprobar que es estable, y el hash de un documento en otra sesión para comprobar que **no** lo es (da 302). De ahí sale la regla de descargar siempre desde una ficha recién abierta.
- Las respuestas reales quedaron versionadas como fixtures: `landing.html` (46 KB), `detail-process.html` (106 KB), `document-viewer.html` (52 KB) y `search-results.xml` (35 KB). No son HTML inventado para que el test pase.
- La bandera `SAVE_RAW=1` guarda cada respuesta cruda en `output/raw/`: es la herramienta con la que se hizo esa exploración y sigue disponible para diagnosticar.

## 5 ¿Cómo enfrentaste JSF / JavaServer Faces?

**Respuesta corta:** dejé de tratarlo como un sitio web y lo traté como un **protocolo con estado**. JSF 1.2 no navega por URL: cada acción es un POST del formulario completo con el `ViewState` vigente, y la respuesta es un XML que dice qué fragmentos del DOM reemplazar. Así que mantengo un documento vivo en memoria y lo **parcheo igual que haría el navegador**.

### Los cinco obstáculos y cómo se resuelve cada uno

| OBSTÁCULO                                              | SOLUCIÓN                                                                                                                                                                                                                                                             |
|--------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| No hay URL a la que ir                                 | POST de <b>todo</b> el formulario <code>fpp</code> serializado en orden de documento, más <code>AJAXREQUEST=_viewRoot</code> , el control «pulsado» y <code>AJAX:EVENTS_COUNT=1</code> .                                                                             |
| El estado vive en el servidor                          | <code>javax.faces.ViewState</code> se lee de la página, viaja en cada POST y se refresca desde <code>#ajax-view-state</code> de cada respuesta. Si expira: <code>SessionExpiredError</code> → sesión nueva.                                                          |
| La respuesta no es una página                          | Es XML A4J con <code>&lt;meta name="Ajax-Update-Ids"&gt;</code> . <code>applyA4jResponse()</code> reemplaza esos ids en el documento vivo (cheerio).                                                                                                                 |
| Los ids <code>j_id244</code> cambian entre despliegues | <b>Nada</b> se localiza por id completo: todo por <b>sufijo</b> ( <code>[id\$=":processoEvento"]</code> ) y los ids concretos se leen del HTML en tiempo de ejecución.                                                                                               |
| El botón visible no dispara la búsqueda                | Su <code>onclick</code> es <code>return ejecutarReCaptcha(); ;A4J.AJAX.Submit(...)</code> : el <code>return</code> corta antes. El submit real es el <code>a4j:jsFunction</code> <code>ejecutarPesquisa</code> , cuyo control se extrae de la página en cada sesión. |

**Extras del mismo mundo:** HTML en ISO-8859-1 y XML en UTF-8 (decodificación por charset declarado); tarro de cookies a mano porque Node no trae uno; y la paginación de movimentações, que es un `rich:inputNumberSlider` con su propio formulario y su propia región Ajax (`containerId`), no un paginador.

#### LA FRASE QUE REMATA LA RESPUESTA

«El reCAPTCHA está en la página pero compilado a `if (false)`: no hay captcha que resolver ni que evadir. Lo difícil de este portal no era la protección, era el protocolo.»

## 6 ¿Por qué ese id? ¿Y si colisiona? ¿Cómo evitas duplicados?

**Respuesta corta:** el id es `PAÍS-FUENTE-<clave única de la fuente>` → `BR-TRF5-0800041-77.2020.4.05.8302`.

La clave es el **número CNJ**, que ya es único a nivel nacional y lo asigna el propio poder judicial; no invento un id, adopto el que la fuente ya garantiza. Y no dependo de que no colisione: el almacenamiento **fusiona** por id en vez de sobrescribir.

### Por qué esa forma

- **Es estable entre corridas.** Un id derivado de la posición en la lista, o un autoincremento, cambiaría al reordenarse los resultados y rompería la reanudación.
- **Es legible y rastreable.** Con el id en la mano se reconstruye la URL de la ficha y se busca el proceso en cualquier sistema judicial brasileño.
- **El prefijo separa espacios de nombres.** `BR-TRF5` es la **fuentes donde se extrajo**, no el tribunal que emitió el número. En el corpus hay procesos cuyo CNJ pertenece a tribunales estaduais (`...8.15.0351` es TJPB, `...8.20.0124` es TJRN) y aun así se listan en la consulta del TRF5. Si mañana se raspa otro tribunal, sus registros no chocan con estos, y que el mismo caso aparezca en dos fuentes es información, no un duplicado.
- **Los documentos heredan:** `<id del proceso>-DOC-<idProcesoDocumento>`, usando el id interno del portal.

### ¿Y si el número no existe?

Los procesos en **segredo de justiça** se listan sin número. En vez de saltarlos o de inventar un id, uso la clave con la que el propio portal los abre: `BR-TRF5-ca-<hash 48 hex>`, verificado estable entre sesiones. Se pierde la legibilidad, no la identidad.

### ¿Qué pasa si colisiona?

- **Un id repetido no destruye nada:** `upsert()` lee el registro existente y **fusiona**. Los campos nuevos definidos ganan, pero `firstSeenAt` se conserva, los documentos se fusionan por su propio id y **una descarga completada nunca se degrada** a pendiente.
- **El riesgo real que sí existe** y conviene decir tú mismo: si un proceso se lista primero sin número (segredo) y después con número, generaría **dos** ids para el mismo proceso. Es un duplicado, no una colisión: el hash `ca` permitiría detectarlo. No lo he observado en la muestra y hoy no está resuelto.
- El caso inverso —dos procesos distintos con el mismo CNJ— exigiría que el poder judicial brasileño emitiera un número duplicado. Si ocurriera, el resultado sería un registro fusionado con dos historiales mezclados, no una pérdida silenciosa.

### Cómo evito duplicados

1. **Por construcción:** el corpus es un mapa indexado por id, no una lista. Un fichero por id en `output/processes/`, más `index.json`.
2. **Los solapes son inevitables y esperados.** La partición binaria vuelve a listar procesos ya vistos; en el log se ve como `30 rows (0 new)`. `upsert()` devuelve `false` y el contador de «nuevos» no se mueve.
3. **Re-listar no deshace trabajo:** se conservan `detailFetched` y `foundInRange` del registro existente. Si no, cada solape forzaría volver a bajar la ficha completa.
4. **El shard manda sobre el índice:** `get()` lee del disco aunque el índice en memoria no conozca el id. Así un índice atrasado nunca provoca que se sobrescriba un shard en vez de fusionarlo.

**TESTS** `store.test.ts` fija que una descarga completada nunca se degrada y que un éxito posterior borra el error anterior. `text.test.ts` fija la construcción de los ids.

## 7 Enséñame los reintentos en el código

**Respuesta corta:** están en un solo sitio, `src/util/retry.ts`. Tres piezas: una **taxonomía** (`isRetryable`), una **política** (`backoffMs`) y un **bucle** (`withRetry`). El resto del código no decide si reintentar: solo etiqueta el error y pasa un `label` y un hook de recuperación.

### Los tres sitios donde se usa, y qué cambia en cada uno

| LLAMADA                           | HOOK DE RECUPERACIÓN                                                                   | PARTICULARIDAD                                                                                                            |
|-----------------------------------|----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| <code>search &lt;rango&gt;</code> | reabrir sesión solo si <code>SessionExpiredError</code> o <code>WafBlockedError</code> | La sesión se recicla también si superó las 400 peticiones.                                                                |
| <code>detail &lt;id&gt;</code>    | igual                                                                                  | Al agotarse, el proceso se deja para <code>retry-failed</code> y no se reintenta en cada reanudación.                     |
| <code>document &lt;id&gt;</code>  | igual, y además <b>relee los enlaces</b> de una ficha nueva antes de repetir           | Los enlaces de documento caducan con la sesión: reintentar con el enlace viejo está condenado.                            |
| <code>movements p&lt;N&gt;</code> | —                                                                                      | <code>retryIf</code> <b>veta</b> el reintento ante sesión expirada: la página N necesita el ViewState que acaba de morir. |

Es la parte más testeada del proyecto: **15 de los 56 tests** están en `retry.test.ts`, con timers falsos para que las esperas de segundos se completen al instante. Cubren clasificación, calendario exponencial, `Retry-After`, techo, agotamiento y el caso del hook de recuperación que falla él mismo.

## 8 ¿Cómo detectas y manejas el 429 en concreto?

**Respuesta corta:** lo detecto en la capa HTTP, antes de que nadie parsee nada: `classify()` convierte 429/5xx/408 en `HttpRetryableError` con el `Retry-After` ya parseado. De ahí en adelante nadie vuelve a mirar un status.

- `parseRetryAfter()` entiende las dos formas del estándar: segundos (`Retry-After: 120`) y fecha HTTP.
- Si el servidor lo manda, **manda él**, incluso por encima del techo de 120 s del backoff propio; solo hay un tope de cordura de 15 minutos para valores absurdos.
- Prevención, no solo reacción: pausa mínima de 700 ms + hasta 300 ms de jitter **entre todas** las peticiones, una sola conexión secuencial, sin concurrencia, y reciclado de sesión cada 400 peticiones.
- En la corrida real, con esos delays, **no apareció un solo 429**. Los fallos observados fueron todos `errorUnexpected.seam`, que es otra cosa: el propio portal fallando en procesos concretos.

## 9 ¿Cómo sabes que cubriste todo el sitio?

**Respuesta corta:** por construcción de la partición, no por confianza. Un rango solo se marca completado cuando su consulta devolvió **menos** que el tope, lo que significa que el portal mostró todas sus filas. Lo que no se puede probar, se declara.

- `state.json.completedRanges` es la prueba positiva: intervalos cerrados por debajo del tope, fusionados por `mergeRanges()` para que no crezcan sin control.
- `state.json.saturatedDays` es la prueba negativa: dónde la interfaz pública no deja llegar. En 35 años de corpus apareció **uno**, y aun así quedó cubierto por clase judicial.
- El total medido (106.763) es el **contraste**, no la prueba: es lo que el portal anuncia.
- Una consulta que devolvió un mensaje en vez de una tabla nunca completa su rango, así que un fallo transitorio no puede convertirse en un hueco permanente.

## 10 ¿Por qué sin navegador?

**Respuesta corta:** porque lo pedía el enunciado, y porque una vez entendido el protocolo el navegador no aportaba nada: coste enorme por proceso y ninguna capacidad extra.

- Cada ficha es **1 GET** más algunos POST de paginación. Un navegador ejecutaría además todo el JavaScript, las hojas de estilo y las imágenes de una página de ~100 KB.
- El único JavaScript que importa —`executarPesquisa`, los parámetros del slider— se lee como texto con una expresión regular. No hace falta ejecutarlo, hace falta entenderlo.
- Sin captcha operativo, la única razón fuerte para un navegador desaparece.
- A escala de 106.763 procesos, un navegador por proceso multiplica el tiempo y la memoria por un orden de magnitud. La cortesía con el servidor ya impone el ritmo; no hace falta añadir peso propio.

## 11 ¿Cuánto tarda el corpus completo? ¿Por qué no paralelizas?

**Respuesta corta:** con los delays por defecto, el descubrimiento son unas 2–3 horas y el corpus entero de fichas + PDF son **semanas** de reloj. Es una decisión, no una limitación: el cuello de botella es la cortesía con el servidor, y paralelizar solo lo convertiría en un problema de rate limiting.

| MEDICIÓN REAL                                             | EXTRAPOLACIÓN AL CORPUS                                                          |
|-----------------------------------------------------------|----------------------------------------------------------------------------------|
| ~1,1 s por búsqueda; ~8 procesos/s en descubrimiento      | 10.000 procesos en ~20 min; el corpus completo en 2–3 h (≈4.000–8.000 búsquedas) |
| ~6,7 s por ficha completa (incluye paginar movimentações) | 106.763 fichas ≈ <b>8 días</b>                                                   |
| ~9 documentos por proceso, ~1,3 s por PDF                 | ≈960.000 PDF ≈ <b>14 días</b> adicionales                                        |

**Por qué un solo hilo:** (1) el enunciado pide delays entre peticiones; (2) el portal está tras un WAF que reacciona a patrones agresivos; (3) el `Store` es de un solo proceso — `index.json` y `state.json` son «último que escribe gana» — así que dos corridas sobre el mismo `output/` se corromperían mutuamente.

**Cómo lo paralelizaría si hiciera falta:** particionar por rango de fechas entre trabajadores con `OUTPUT_DIR` distinto y fusionar por id al final, que es exactamente lo que permite tener ids estables.

## 12 ¿Qué pasa si el portal cambia el HTML?

**Respuesta corta:** falla ruidosamente y en un solo sitio. Los parsers lanzan `UnexpectedStructureError`, que **nunca se reintent**a, y todo el conocimiento del portal está aislado en `src/pje/`.

- Selectores **por sufijo**, no por id completo: sobrevive a que cambien los `j_idNNN`, que es el cambio más frecuente.
- Los ids volátiles se leen en tiempo de ejecución: el control de búsqueda, el del slider, el `containerId`.
- Los mensajes de error dicen **qué** no encontraron (`form "fPP" not found`, `search response did not render the results grid (updated: ...)`), así que el diagnóstico no empieza de cero.
- Las fixtures son capturas reales: cuando el portal cambie, se recaptura y el test dice exactamente qué se rompió.

### 13 ¿Cómo pruebas un scraper sin depender de la red?

**Respuesta corta:** 56 tests, 8 suites, **cero red**, sobre fixtures que son respuestas reales del portal guardadas tal cual.

- **Parsers** contra HTML real de 46–106 KB: tope de 30 filas, banner de desborde, secreto de justiça sin número, partes con abogados indentados, slider de movimentações, las dos rutas de descarga.
- **Protocolo A4J**: serialización del formulario, construcción del cuerpo, parcheo del DOM con la respuesta XML.
- **Política de reintentos** con timers falsos: las esperas de 30 s se completan al instante.
- **Funciones puras**: fechas, `splitRange`, ids, nombres de fichero, fusión de rangos y de documentos, rename con reintento.
- Lo que **no** se puede testear offline (que el portal siga respondiendo como el 2026-08-25) se cubre con la corrida en vivo y su log.

### 14 ¿Qué NO cubre tu solución?

**Respuesta corta:** cuatro huecos, todos conocidos y documentados. Decirlos tú antes de que los encuentren vale más que fingir que no existen.

1. **Días saturados.** Si un mismo día + clase judicial supera las 30 filas, ese resto es inalcanzable **por la interfaz pública**. Queda anotado en `state.json`.
2. **Partes paginadas.** Las tres tablas de partes tienen su propio `dataScroller`. Se lee la página que la ficha renderiza; un proceso con más partes que una página quedaría truncado. No se ha observado en la muestra (las movimentações sí se paginan completas).
3. **Documentos sin ruta pública** (enlace `about:blank`): se registran como `unavailable`, que es distinto de `failed`.
4. `reportPDF.seam` (expediente completo, botón «Imprimir») devuelve 302 fuera del flujo del navegador. No se usa: los PDF se obtienen documento a documento.

Y el detalle de la sección 8.3: `npm run status` puede quedarse corto en sus contadores mientras hay una corrida viva, porque lee `index.json` y no los shards.

### 15 ¿Y el alcance legal / ético?

**Respuesta corta:** la Consulta Pública es de acceso libre y sin autenticación, el organizador del desafío declara que ya posee esta información y que el scraper es una prueba técnica. No hay credenciales, no hay evasión de protección y los delays son conservadores por defecto.

- No se resuelve ni se evade ningún captcha: está desactivado en el servidor.
- No se falsifican credenciales ni se accede a nada tras un login.
- `output/` está en `.gitignore`; solo se versiona una muestra de PDF como evidencia de la descarga de punta a punta —resoluciones publicadas por el propio tribunal para consulta pública—. El resto de los datos no se sube.
- Los delays y el reciclado de sesión existen para no degradar el servicio de un portal judicial.



# 11 · Chuleta final

Lo que hay que tener en la cabeza cinco minutos antes de entrar.

## Las cifras

| DATO                                            | VALOR                                                                                                       |
|-------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| Corpus anunciado por el portal                  | <b>106.763</b> procesos (medido 2026-08-26, 1 petición, sin ingesta)                                        |
| Tope de filas por consulta                      | <b>30</b> , y el paginador se renderiza vacío                                                               |
| Movimentações por página                        | <b>15</b> , paginadas con un <code>rich:inputNumberSlider</code>                                            |
| Líneas de TypeScript / tests                    | <b>3.432</b> líneas · <b>56</b> tests · <b>8</b> suites · sin red                                           |
| Dependencias de ejecución                       | <b>2</b> : <code>axios</code> y <code>cheerio</code>                                                        |
| Intentos por elemento                           | <b>3</b> ( <code>MAX_ATTEMPTS</code> )                                                                      |
| Backoff                                         | 2 s · 2 <sup>n</sup> con jitter completo, techo <b>120 s</b> ; <code>Retry-After</code> manda (tope 15 min) |
| Pausa del WAF / de <code>errorUnexpected</code> | <b>90 s / 5 s</b>                                                                                           |
| Cortesía entre peticiones                       | <b>700 ms + hasta 300 ms</b> de jitter; sesión reciclada cada <b>400</b> peticiones                         |
| Ritmo medido                                    | ~1,1 s por búsqueda · ~8 procesos/s descubriendo · ~6,7 s por ficha completa                                |
| Estado de la corrida en la guía                 | 10.062 procesos · 123 fichas · 1.101 documentos · 11 PDF · 5 fallos                                         |

## Los comandos

```
PowerShell

# las dos fases, en orden - reanudable, se puede cortar con Ctrl+C
npm run all

# fases por separado
npm run discover      fase 1: enumera particionando por fecha
npm run download      fase 2: fichas completas + PDFs

# operación
npm run status        progreso desde los ficheros de salida, SIN red
npm run retry--failed reprocesa lo anotado en output/failed.json
npm test              56 tests offline con fixtures reales
npm run typecheck      tsc --noEmit sobre src y sobre los tests

# demostración acotada de ~2 minutos (bash)
DATE_FROM=1990-01-01 DATE_TO=1994-12-31 MAX_PROCESSES=3 MAX_DOCUMENTS=4 npm run all

# lo mismo en PowerShell: las variables se fijan antes
$env:MAX_DOCUMENTS='4'; npm run all

# pasada de solo fichas, sin descargar PDFs
$env:SKIP_DOWNLOADS='1'; npm run download
```

## Las variables que más se usan

| VARIABLE                      | POR DEFECTO       | EFEECTO                                                    |
|-------------------------------|-------------------|------------------------------------------------------------|
| DATE_FROM / DATE_TO           | 1985-01-01 / hoy  | Ventana que particiona la fase 1                           |
| MAX_DISCOVERED                | 0 = sin límite    | Detiene la fase 1 al tener N procesos almacenados          |
| MAX_PROCESSES / MAX_DOCUMENTS | 0 / 0             | Topes de la fase 2 por ejecución                           |
| SKIP_DOWNLOADS                | off               | Fase 2 sin PDFs: fichas completas, descargas para después  |
| MAX_ATTEMPTS                  | 3                 | Intentos por petición antes de anotar el fallo y seguir    |
| MIN_DELAY_MS / JITTER_MS      | 700 / 300         | Cortesía entre peticiones                                  |
| SAVE_RAW / DEBUG              | off / off         | Guardar cada respuesta cruda / trazar cada petición        |
| OUTPUT_DIR                    | output            | Carpeta de salida (usar otra para una corrida en paralelo) |
| PJE_BASE_URL                  | pjett.trf5.jus.br | El mismo PJe corre en otros tribunales                     |

#### PRECAUCIÓN OPERATIVA

El `Store` es de un solo proceso: `index.json` y `state.json` son «último que escribe gana». **Nunca lances dos corridas contra el mismo `output/`.** Antes de arrancar, comprueba que no haya un `ts-node src/index.ts` vivo (`Get-Process node`). Matar el proceso de golpe es seguro: los shards ya están en disco y `reconcileShards()` recupera lo que el índice no alcanzó a registrar.

## Las siete frases para memorizar

- Volumen:** «Lo medí en frío, sin ingesta: una petición con una fecha inexistente que el conversor descarta, y el portal declara 106.763.»
- Paginación:** «No hay paginador. Particiono el espacio de búsqueda por fecha hasta que cada tramo cabe bajo el tope de 30.»
- JSF:** «No es un sitio web, es un protocolo con estado. Mantengo un documento vivo y lo parcheo con los fragmentos que devuelve el servidor, igual que el navegador.»
- Reintentos:** «Clasifico primero, reacciono después. Exponencial con jitter para lo transitorio, pausa larga y sesión nueva para el WAF, fatal sin reintento para lo que no va a cambiar.»
- Escalado:** «Bajé de 5 intentos a 3 porque medí que ese error es determinista por proceso, no rate limiting. El 429 conserva la curva completa.»
- Identidad:** «No invento el id: adopto el número CNJ, que ya es único a nivel nacional, con prefijo de fuente. Y no dependo de que no colisione: el store fusiona por id.»
- PDF:** «Valido status, firma `%PDF-` y tamaño, y escribo a `.part` antes de renombrar. Nunca queda basura con extensión de PDF.»

#### SI SOLO TE ACUERDAS DE UNA COSA

**Nunca completar algo en silencio.** Rango no cerrado si la respuesta no fue una tabla de resultados. `detailFetched: false` si el historial quedó truncado. Día saturado anotado con su motivo. Documento fallido registrado con su error. Abortar en vez de arrancar de cero si el estado está corrupto. Es la misma decisión repetida en cinco capas, y es lo que separa un scraper que produce datos de uno que produce datos *en los que se puede confiar*.